Assignment - 2

Water Jug Problem

Title: Development of a water Jug problem in Python.

Objective: The goal of this assignment is a to implement the water jug problem using Breadth First Search(BFS) algorithm in python. The problem involves two jugs with given capacities and determining a sequence of operations to measure an exact amount of water amount of water.

Problem Statement:

You are given two jugs:
- Jug1 with capacity A liters.
- Jug2 with capacity B liters.
- A target amount of water C liters [where C ≤ max(A, B)]

Using the following operations:
1. Fill a jug completely.
2. Empty a jug completely.
3. Pour water from one jug to another until either the receiving jug is full or the pouring jug is empty.

Your task is to find the sequence of operation to measure exactly (liters using the jug's.

Algorithm Approach:

Steps:

1. Define a stat's structure
   - Represent the amount of water in each jug a [X, Y] pair.

2. Use a queue for BFS traversal
   - Strat from initial states.
   - Use a queue to explore possible stat's

3. Track Visited stat's.
   - Maintain set of visited states to present cycles.

4. Check for target condition:
   - If either jug contains exactly (litre print the sequence of steps and exist.

5. Generate possible Next stat's.
   - Fill Jug [A, Y]
   - Fill Jug [X, B]
   - Empty Jug1 [0, y]
   - Empty Jug2 [X, 0]
   - Pour water from Jug1 to Jug2
   - Pour water from Jug2 to Jug1

6 Continue BFS until a selection is found.

Expected Output:

For example, it

Jug1 capacity = 4l
Jug2 capacity = 3L
Target = 3L

```java
import java.util.*;
class WaterJugSolver {
    static class State {
        int jug1, jug2;
        List<String> operations;
        State(int a, int b, List<String> ops) {
            this.jug1 = a;
            this.jug2 = b;
            this.operations = new ArrayList<>(ops);
        }
    }
    public static void solveWaterJug(int A, int B, int C) {
        if (C > Math.max(A, B)) {
            System.out.println("No solution possible.");
            return;
        }
        Queue<State> queue = new LinkedList<>();
        Set<String> visited = new HashSet<>();
        List<String> initialOps = new ArrayList<>();
        initialOps.add("Start: (0,0)");
        queue.add(new State(0, 0, initialOps));
        visited.add("0,0");
        while (!queue.isEmpty()) {
            State current = queue.poll();
            int a = current.jug1, b = current.jug2;
            if (a == C || b == C) {
                System.out.println("Solution found!\n");
                for (String step : current.operations) {
                    System.out.println(step);
                }
                return;
            }
```

```java
            List<State> nextStates = Arrays.asList(
                new State(A, b, addOperation(current.operations, "Fill Jug 1 (" + A + "," + b + ")")),
                new State(a, B, addOperation(current.operations, "Fill Jug 2 (" + a + "," + B + ")")),
                new State(0, b, addOperation(current.operations, "Empty Jug 1 (0," + b + ")")),
                new State(a, 0, addOperation(current.operations, "Empty Jug 2 (" + a + ",0)")),
                new State(a - Math.min(a, B - b), b + Math.min(a, B - b),
                        addOperation(current.operations, "Pour Jug1 -> Jug2 (" + (a - Math.min(a, B - b)) + "," + (
                        + Math.min(a, B - b)) + ")")),
                new State(a + Math.min(b, A - a), b - Math.min(b, A - a),
                        addOperation(current.operations, "Pour Jug2 -> Jug1 (" + (a + Math.min(b, A - a)) + "," + (
                        - Math.min(b, A - a)) + ")"))
            for (State nextState : nextStates) {
                String stateKey = nextState.jug1 + "," + nextState.jug2;
                if (!visited.contains(stateKey)) {
                    visited.add(stateKey);
                    queue.add(nextState);
                }}System.out.println("No solution found.");
    }
    private static List<String> addOperation(List<String> operations, String newOp) {
        List<String> newOps = new ArrayList<>(operations);
        newOps.add(newOp);
        return newOps;
    }
    [illegible]
    public static void main(String[] args) {
        int A = [illegible], B = [illegible], C = [illegible];
        solveWaterJug(A, B, C);
    }
```

Output:

Conclusion:

We have successfully developed a program to implement the Water Jug Problem

C

21/2/25